

SmartPyME

Documento de Contrato API

Alvaro Raul Quispe Condori

Junio 2026

Índice

1	Información general	7
2	Objetivo	8
3	Convenciones	8
3.1	Versionado	8
3.2	Convención de rutas	8
3.3	Convención JSON	9
4	Seguridad	10
4.1	Autenticación	10
4.2	Autorización	10
5	Objetos compartidos	11
5.1	Request DTOs	11
5.1.1	RegisterRequest	11
5.1.2	LoginRequest	12
5.1.3	CreateCategoryRequest	12
5.1.4	UpdateCategoryRequest	12
5.1.5	CreateProductRequest	12
5.1.6	UpdateProductRequest	12
5.1.7	CreateVariantRequest	13
5.1.8	UpdateVariantRequest	13
5.1.9	AddCartItemRequest	13
5.1.10	UpdateCartItemRequest	14
5.1.11	CheckoutRequest	14
5.1.12	ShippingAddress (objeto anidado)	14
5.1.13	ConfirmPaymentRequest	15
5.1.14	UpdateOrderStatusRequest	15
5.1.15	UpdateStockRequest	15

5.1.16	CreateStockMovementRequest	15
5.2	Response DTOs	16
5.2.1	AuthResponse	16
5.2.2	User	16
5.2.3	Category	16
5.2.4	ProductVariant	17
5.2.5	ProductImage	17
5.2.6	Product	17
5.2.7	ProductListItem (versión resumida para listados)	18
5.2.8	CartItem	19
5.2.9	Cart	19
5.2.10	OrderItem	20
5.2.11	Payment	20
5.2.12	Order	21
5.2.13	CheckoutResponse	21
5.2.14	StockMovement	22
5.2.15	StockAlert	22
5.2.16	DashboardMetrics	23
5.2.17	TopProduct	23
5.2.18	OrdersByStatus	24
5.2.19	SalesTimelinePoint	24
5.3	Objetos comunes	24
5.3.1	Pagination	24
5.3.2	SuccessResponse (envelope)	24
5.3.3	ErrorResponse (envelope)	25
5.3.4	PaginatedResponse	25
5.4	Enumeraciones	26
5.4.1	user_role	26
5.4.2	order_status	26
5.4.3	payment_status	26
5.4.4	payment_method	27
5.4.5	currency	27
5.4.6	cart_status	27
5.4.7	stock_movement_type	27
5.4.8	stock_alert_status	28
6	Formato estándar	28
6.1	Respuesta exitosa	28
6.2	Respuesta con error	29
6.3	Respuesta paginada	29
7	Endpoints	30
7.1	Health Check	30
7.1.1	[API-001] Health Check	30
7.1.2	Códigos HTTP	30

7.1.3	Response	31
7.2	Autenticación	31
7.2.1	[API-002] Registro de usuario	31
7.2.2	Request Body	31
7.2.3	Response 201	31
7.2.4	Códigos HTTP	32
7.2.5	Reglas	32
7.2.6	[API-003] Inicio de sesión	32
7.2.7	Request Body	32
7.2.8	Response 200	33
7.2.9	Códigos HTTP	33
7.2.10	Reglas	33
7.2.11	[API-004] Usuario actual	33
7.2.12	Headers	33
7.2.13	Response 200	34
7.2.14	Códigos HTTP	34
7.3	Categorías (admin)	34
7.3.1	[API-005] Listar categorías	34
7.3.2	Query Params	34
7.3.3	Response 200	35
7.3.4	Códigos HTTP	35
7.3.5	[API-006] Crear categoría	35
7.3.6	Request Body	35
7.3.7	Response 201	35
7.3.8	Códigos HTTP	35
7.3.9	Reglas	36
7.3.10	[API-007] Obtener categoría por ID	36
7.3.11	Path Params	36
7.3.12	Response 200: Category · 404 si no existe.	36
7.3.13	[API-008] Actualizar categoría	36
7.3.14	Path Params: id (uuid)	37
7.3.15	Request Body: UpdateCategoryRequest	37
7.3.16	Response 200: Category	37
7.3.17	Códigos HTTP	37
7.3.18	[API-009] Eliminar categoría	37
7.3.19	Path Params: id (uuid)	37
7.3.20	Response 204: Sin contenido.	37
7.3.21	Códigos HTTP	37
7.4	Productos (admin)	38
7.4.1	[API-010] Listar productos (admin)	38
7.4.2	Query Params	38
7.4.3	Response 200: PaginatedResponse<ProductListItem>	38
7.4.4	[API-011] Crear producto	39
7.4.5	Request Body: CreateProductRequest	39
7.4.6	Response 201: Product (con arrays vacíos de variants e images)	39

7.4.7	Códigos HTTP: 201, 400, 401, 403, 404 (categoría no existe)	39
7.4.8	Reglas	39
7.4.9	[API-012] Obtener producto por ID	39
7.4.10	Response 200: Product (completo con variants e images)	39
7.4.11	Códigos HTTP: 200, 404	39
7.4.12	[API-013] Actualizar producto	39
7.4.13	Request Body: UpdateProductRequest	40
7.4.14	Response 200: Product	40
7.4.15	Códigos HTTP: 200, 400, 404	40
7.4.16	[API-014] Eliminar producto	40
7.4.17	Response 204: Sin contenido	40
7.4.18	Códigos HTTP: 204, 404	40
7.4.19	Reglas	40
7.5	Variantes de producto	40
7.5.1	[API-015] Crear variante	40
7.5.2	Path Params: id (uuid del producto)	40
7.5.3	Request Body: CreateVariantRequest	40
7.5.4	Response 201: ProductVariant	40
7.5.5	Códigos HTTP: 201, 400, 404, 409 (combinación duplicada)	40
7.5.6	Reglas	40
7.5.7	[API-016] Actualizar variante	41
7.5.8	Path Params: id (producto), variantId (variante)	41
7.5.9	Request Body: UpdateVariantRequest	41
7.5.10	Response 200: ProductVariant	41
7.5.11	Códigos HTTP: 200, 400, 404, 409	41
7.5.12	[API-017] Eliminar variante	41
7.5.13	Response 204	41
7.5.14	Códigos HTTP: 204, 404, 409 (variante con order_items)	41
7.6	Imágenes de producto	41
7.6.1	[API-018] Subir imagen	41
7.6.2	Request	42
7.6.3	Query Params	42
7.6.4	Response 201: ProductImage	42
7.6.5	Códigos HTTP	42
7.6.6	Reglas	42
7.6.7	[API-019] Eliminar imagen	42
7.6.8	Response 204	42
7.7	Catálogo público	43
7.7.1	[API-020] Listar productos del catálogo	43
7.7.2	Query Params	43
7.7.3	Response 200: PaginatedResponse<ProductListItem>	43
7.7.4	Reglas	43
7.7.5	[API-021] Detalle de producto público	44
7.7.6	Path Params: slug (string)	44
7.7.7	Response 200: Product (completo)	44

7.7.8	Códigos HTTP: 200, 404	44
7.7.9	Reglas	44
7.8	Carrito de compras	44
7.8.1	[API-022] Obtener carrito activo	44
7.8.2	Headers (opcional para guest)	44
7.8.3	Response 200: Cart	45
7.8.4	Códigos HTTP: 200, 500	45
7.8.5	Reglas	45
7.8.6	[API-023] Agregar item al carrito	45
7.8.7	Request Body: AddCartItemRequest	45
7.8.8	Response 200: Cart (con el nuevo item)	45
7.8.9	Códigos HTTP	45
7.8.10	Reglas	45
7.8.11	[API-024] Actualizar cantidad de un item	46
7.8.12	Path Params: id (uuid del cart_item)	46
7.8.13	Request Body: UpdateCartItemRequest	46
7.8.14	Response 200: Cart	46
7.8.15	Códigos HTTP: 200, 404, 422 (stock insuficiente)	46
7.8.16	[API-025] Eliminar item del carrito	46
7.8.17	Response 200: Cart	46
7.8.18	Códigos HTTP: 200, 404	46
7.8.19	[API-026] Vaciar carrito	46
7.8.20	Response 200: Cart (vacío)	46
7.9	Checkout y pagos	47
7.9.1	[API-027] Iniciar checkout	47
7.9.2	Request Body: CheckoutRequest	47
7.9.3	Response 200: CheckoutResponse	47
7.9.4	Códigos HTTP	47
7.9.5	Reglas	47
7.9.6	[API-028] Confirmar pago	47
7.9.7	Request Body: ConfirmPaymentRequest	48
7.9.8	Response 200: Order (actualizado con status='pagado' o fallido)	48
7.9.9	Códigos HTTP	48
7.9.10	Reglas	48
7.9.11	[API-029] Webhook de Stripe (reservado v2)	48
7.9.12	Request: Payload de Stripe	49
7.9.13	Response 200: { received: true }	49
7.9.14	Códigos HTTP: 200, 400 (firma inválida)	49
7.10	Pedidos	49
7.10.1	[API-030] Listar pedidos (admin)	49
7.10.2	Query Params	49
7.10.3	Response 200: PaginatedResponse<Order>	49
7.10.4	Códigos HTTP: 200, 401, 403	49
7.10.5	[API-031] Obtener pedido por ID	50
7.10.6	Response 200: Order	50

7.10.7	Códigos HTTP: 200, 401, 403 (no es dueño ni admin), 404	50
7.10.8	Reglas	50
7.10.9	[API-032] Cambiar estado de pedido (admin)	50
7.10.10	Request Body: UpdateOrderStatusRequest	50
7.10.11	Response 200: Order	50
7.10.12	Códigos HTTP	50
7.10.13	Reglas	50
7.10.14	[API-033] Historial de pedidos del cliente	51
7.10.15	Query Params: page, limit, status (opcional)	51
7.10.16	Response 200: PaginatedResponse<Order>	51
7.10.17	Códigos HTTP: 200, 401	51
7.10.18	Reglas	51
7.11	Inventario (admin)	51
7.11.1	[API-034] Listar alertas de stock	51
7.11.2	Query Params: page, limit, status (default active)	51
7.11.3	Response 200: PaginatedResponse<StockAlert>	51
7.11.4	Códigos HTTP: 200, 401, 403	51
7.11.5	[API-035] Listar movimientos de stock	51
7.11.6	Query Params: page, limit, type, product_variant_id, from, to	52
7.11.7	Response 200: PaginatedResponse<StockMovement>	52
7.11.8	Códigos HTTP: 200, 401, 403	52
7.11.9	[API-036] Registrar movimiento manual de stock	52
7.11.10	Request Body: CreateStockMovementRequest	52
7.11.11	Response 201: StockMovement	52
7.11.12	Códigos HTTP: 201, 400, 404, 422 (stock negativo)	52
7.11.13	Reglas	52
7.12	Dashboard (admin)	52
7.12.1	[API-037] Métricas generales	52
7.12.2	Query Params	52
7.12.3	Response 200: DashboardMetrics	53
7.12.4	Códigos HTTP: 200, 401, 403	53
7.12.5	Reglas	53
7.12.6	[API-038] Top productos vendidos	53
7.12.7	Query Params: from, to, limit (default 5, max 20)	53
7.12.8	Response 200: TopProduct[]	53
7.12.9	[API-039] Pedidos por estado	53
7.12.10	Query Params: from, to	54
7.12.11	Response 200: OrdersByStatus[]	54
7.12.12	[API-040] Serie temporal de ventas	54
7.12.13	Query Params	54
7.12.14	Response 200: SalesTimelinePoint[]	54
7.13	Resumen de endpoints	54

8 Códigos de error

55

9	Versionado	56
10	Historial de cambios	56
11	Riesgos técnicos	57
12	Criterios de aceptación	58

1 Información general

Proyecto: SmartPyME — Plataforma Digital Móvil para PYMES de Comercio Minorista

Versión: 1.0.0

Fecha: Junio 2026

Protocolo: HTTPS (HTTP solo en desarrollo local)

Formato de intercambio: JSON (UTF-8)

Versión de la API: v1 (sin versionado en URL; versionado por tags de Git)

Base URL: `https://api.smartpyme.local/api` (desarrollo: `http://localhost:3000/api`)

Autenticación: JWT (JSON Web Token) en header Authorization: Bearer <token>

Documentación interactiva: `https://api.smartpyme.local/api/docs` (Swagger UI desde OpenAPI 3.1)

Documentos de referencia:

- `ARTICULO.md` — Artículo de revisión bibliométrica.
- `ALCANCE.md` — Documento de alcance del proyecto.
- `BACKLOG.md` — Backlog del producto.
- `MODELOS-DATOS.md` — Modelo de datos y esquema Drizzle.
- Como la información general debe identificar de forma única la versión del contrato.
- Como la versión de la API debe mantenerse compatible con el frontend correspondiente.

- Como este documento debe ser la fuente oficial del contrato entre frontend y backend.

2 Objetivo

Este documento define el contrato de comunicación HTTP/JSON entre la aplicación móvil Flutter (cliente y panel admin) y el backend de SmartPyME. Establece las rutas, métodos, estructuras de datos (DTOs), reglas de validación, códigos de respuesta y mecanismos de autenticación necesarios para garantizar una integración consistente entre todos los componentes del sistema.

- Como este documento debe permitir el desarrollo independiente del frontend y backend.
- Como todos los contratos deben derivarse de los requisitos funcionales de ALCANCE.md.
- Como cualquier modificación del contrato debe reflejarse primero en este documento antes de implementarse.
- Como este documento debe estar sincronizado con MODELOS-DATOS.md ante cualquier cambio estructural.

3 Convenciones

3.1 Versionado

- Sin versionado en URL: la API se mantiene en v1 mientras no haya cambios incompatibles.
- Cambios incompatibles (eliminar campos, renombrar rutas, cambiar tipos) requieren una nueva rama del backend y migración del cliente.
- El versionado se realiza por **tags de Git** del backend (v1.0.0, v1.1.0, etc.).
- Las versiones anteriores se mantienen en ramas separadas durante el ciclo de vida del proyecto.

3.2 Convención de rutas

- Recursos en **plural** (/products, /orders, /categories).
- Solo **sustantivos** en las rutas (sin verbos).

- Verbos HTTP estándar: GET (lectura), POST (creación), PATCH (actualización parcial), PUT (reemplazo total), DELETE (eliminación).
- IDs en path param como UUID: /resources/{id}.
- Sub-recursos anidados: /products/:id/variants, /products/:id/images.
- Endpoints administrativos bajo /api/admin/* o /api/{resource} con verificación de rol admin.
- Endpoints públicos del cliente bajo /api/catalog/* (sin auth) y /api/{resource} (con auth de customer).

```
GET    /api/products
POST   /api/products
GET    /api/products/{id}
PATCH /api/products/{id}
DELETE /api/products/{id}
```

3.3 Convención JSON

- **Nombres de campos:** snake_case en todas las claves (first_name, stripe_payment_intent_id).
- **Fechas:** Unix timestamp en milisegundos (1719496200000). El backend transforma desde/hacia el formato TEXT ISO 8601 de SQLite.
- **UUIDs:** strings de 36 caracteres con guiones (550e8400-e29b-41d4-a716-446655440000).
- **Booleanos:** true / false nativos de JSON.
- **Valores monetarios:** INTEGER en centavos (ej. 2999 = S/ 29.99).
- **Valores nulos:** null explícito cuando el campo es opcional y no tiene valor.
- **Codificación:** UTF-8 en todo el intercambio.
- Como todas las rutas deben seguir la misma convención.
- Como todos los objetos JSON deben mantener el mismo formato.
- Como todas las fechas deben utilizar Unix timestamp en milisegundos.

4 Seguridad

4.1 Autenticación

- **Método:** JWT firmado con HS256.
- **Header requerido:** Authorization: Bearer <jwt>.
- **Claims del token:** sub (user_id), role (admin | customer), email, iat, exp.
- **Tiempo de expiración:** 7 días (604800 segundos).
- **Renovación:** en esta versión no hay refresh token. El cliente debe reautenticarse al expirar.
- **Almacenamiento en cliente:** flutter_secure_storage (cifrado con Keystore en Android y Keychain en iOS).
- **Logout:** el cliente descarta el token localmente (no hay blacklist en backend en v1).

4.2 Autorización

Rol	Permisos
guest (sin token)	Listar productos del catálogo público, ver detalle, gestionar carrito anónimo, checkout.
customer	Todo lo de guest + ver historial de pedidos, gestionar datos personales, crear tickets.
admin	Acceso completo: gestión CRUD de productos/categorías/variantes/imágenes, gestión de pedidos, dashboard, alertas de stock, configuración.

Recurso	guest	customer	admin
GET /api/catalog/*	✓	✓	✓
GET/POST/PATCH/DELETE /api/cart/*	✓ (por session_id)	✓	✓
POST /api/checkout	✓	✓	✓
GET /api/customers/me/orders	✗	✓	✗
GET /api/auth/me	✗	✓	✓

Recurso	guest	customer	admin
POST /api/products	X	X	✓
GET /api/orders (admin)	X	X	✓
PATCH /api/orders/:id/status	X	X	✓
GET /api/dashboard/*	X	X	✓
GET /api/inventory/*	X	X	✓

- Como todos los endpoints protegidos deben validar el JWT antes de cualquier operación.
- Como los permisos deben verificarse según el rol del token antes de ejecutar la acción.
- Como un cliente solo puede acceder a sus propios recursos (carrito, pedidos, perfil).
- Como los endpoints de admin deben rechazar tokens con `role !== 'admin'` con código 403.

5 Objetos compartidos

Todos los endpoints referencian los DTOs definidos en esta sección.

5.1 Request DTOs

5.1.1 RegisterRequest

Campo	Tipo	Obligatorio	Descripción
email	string (email)	Sí	Correo electrónico único.
password	string	Sí	Contraseña en texto plano (mín. 8 caracteres). Se hashea en backend.
name	string	Sí	Nombre completo.
phone	string	No	Teléfono de contacto.
role	string	No	admin o customer. Por defecto customer.

5.1.2 LoginRequest

Campo	Tipo	Obligatorio	Descripción
email	string (email)	Sí	Correo electrónico.
password	string	Sí	Contraseña.

5.1.3 CreateCategoryRequest

Campo	Tipo	Obligatorio	Descripción
name	string	Sí	Nombre único de la categoría.
description	string	No	Descripción opcional.

5.1.4 UpdateCategoryRequest

Campo	Tipo	Obligatorio	Descripción
name	string	No	Nuevo nombre.
description	string	No	Nueva descripción.
is_active	boolean	No	Activar/desactivar.

5.1.5 CreateProductRequest

Campo	Tipo	Obligatorio	Descripción
name	string	Sí	Nombre del producto.
description	string	No	Descripción detallada.
base_price	integer	Sí	Precio base en centavos (> 0).
category_id	string (uuid)	Sí	ID de la categoría.
is_active	boolean	No	Por defecto true.

5.1.6 UpdateProductRequest

Campo	Tipo	Obligatorio	Descripción
name	string	No	Nuevo nombre.
description	string	No	Nueva descripción.
base_price	integer	No	Nuevo precio base (> 0).
category_id	string (uuid)	No	Nueva categoría.
is_active	boolean	No	Activar/desactivar.

5.1.7 CreateVariantRequest

Campo	Tipo	Obligatorio	Descripción
size	string	Sí	Talla (ej. S, M, L, 30, 32).
color	string	Sí	Color (ej. Rojo, Azul).
sku	string	Sí	SKU único.
stock	integer	No	Stock inicial (por defecto 0).
additional_price	integer	No	Precio adicional en centavos (por defecto 0).

5.1.8 UpdateVariantRequest

Campo	Tipo	Obligatorio	Descripción
size	string	No	Nueva talla.
color	string	No	Nuevo color.
sku	string	No	Nuevo SKU.
stock	integer	No	Nuevo stock (>= 0).
additional_price	integer	No	Nuevo precio adicional.

5.1.9 AddCartItemRequest

Campo	Tipo	Obligatorio	Descripción
product_variant_id	string (uuid)	Sí	ID de la variante.
quantity	integer	Sí	Cantidad (> 0).

5.1.10 UpdateCartItemRequest

Campo	Tipo	Obligatorio	Descripción
quantity	integer	Sí	Nueva cantidad (> 0).

5.1.11 CheckoutRequest

Campo	Tipo	Obligatorio	Descripción
shipping_address	object (ShippingAddress)	Sí	Dirección de envío.
guest_email	string (email)	No	Email del invitado (requerido si es guest).
guest_name	string	No	Nombre del invitado.
guest_phone	string	No	Teléfono del invitado.
notes	string	No	Notas del pedido.

5.1.12 ShippingAddress (objeto anidado)

Campo	Tipo	Obligatorio	Descripción
street	string	Sí	Calle y número.
city	string	Sí	Ciudad.
state	string	Sí	Departamento/estado.
zip	string	No	Código postal.
country	string	Sí	Código de país ISO 3166-1 alpha-2 (PE, US).

Campo	Tipo	Obligatorio	Descripción
references	string	No	Referencias de la dirección.

5.1.13 ConfirmPaymentRequest

Campo	Tipo	Obligatorio	Descripción
payment_intent	string	Sí	ID del PaymentIntent devuelto por Stripe.
order_id	string (uuid)	Sí	ID de la orden a confirmar.

5.1.14 UpdateOrderStatusRequest

Campo	Tipo	Obligatorio	Descripción
status	string	Sí	Nuevo estado del pedido.

5.1.15 UpdateStockRequest

Campo	Tipo	Obligatorio	Descripción
stock	integer	Sí	Nuevo stock (≥ 0).

5.1.16 CreateStockMovementRequest

Campo	Tipo	Obligatorio	Descripción
product_variant_id	string (uuid)	Sí	ID de la variante.
type	string	Sí	restock o adjustment.
quantity	integer	Sí	Cantidad (positiva o negativa).
notes	string	No	Notas.

5.2 Response DTOs

5.2.1 AuthResponse

Campo	Tipo	Descripción
token	string	JWT firmado.
user	object (User)	Datos del usuario.
expires_at	integer (timestamp ms)	Fecha de expiración del token.

5.2.2 User

Campo	Tipo	Descripción
id	string (uuid)	ID único.
email	string	Correo electrónico.
name	string	Nombre completo.
phone	string null	Teléfono.
role	string	admin o customer.
is_active	boolean	Estado de la cuenta.
created_at	integer (timestamp ms)	Fecha de creación.

5.2.3 Category

Campo	Tipo	Descripción
id	string (uuid)	ID único.
name	string	Nombre.
slug	string	URL-friendly.
description	string null	Descripción.
is_active	boolean	Estado.
product_count	integer	Total de productos asociados.

Campo	Tipo	Descripción
created_at	integer (timestamp ms)	Fecha de creación.
updated_at	integer (timestamp ms)	Última modificación.

5.2.4 ProductVariant

Campo	Tipo	Descripción
id	string (uuid)	ID único.
product_id	string (uuid)	ID del producto.
size	string	Talla.
color	string	Color.
sku	string	SKU único.
stock	integer	Stock disponible.
additional_price	integer	Precio adicional en centavos.
final_price	integer	Precio final (base + additional) en centavos.
is_in_stock	boolean	true si stock > 0.
created_at	integer (timestamp ms)	Fecha de creación.
updated_at	integer (timestamp ms)	Última modificación.

5.2.5 ProductImage

Campo	Tipo	Descripción
id	string (uuid)	ID único.
url	string	URL pública en R2.
position	integer	Orden (0 = principal).

5.2.6 Product

Campo	Tipo	Descripción
id	string (uuid)	ID único.
name	string	Nombre.
slug	string	URL-friendly.
description	string null	Descripción.
base_price	integer	Precio base en centavos.
category_id	string (uuid)	ID de categoría.
category	object (Category)	Categoría anidada.
is_active	boolean	Estado.
variants	array (ProductVariant)	Variantes del producto.
images	array (ProductImage)	Imágenes ordenadas por position.
main_image	object (ProductImage) null	Imagen con menor position.
min_price	integer	Precio mínimo entre todas las variantes en stock.
total_stock	integer	Suma del stock de todas las variantes.
has_stock	boolean	true si total_stock > 0.
created_at	integer (timestamp ms)	Fecha de creación.
updated_at	integer (timestamp ms)	Última modificación.

5.2.7 ProductListItem (versión resumida para listados)

Campo	Tipo	Descripción
id	string (uuid)	ID único.
name	string	Nombre.

Campo	Tipo	Descripción
slug	string	URL-friendly.
base_price	integer	Precio base en centavos.
main_image_url	string null	URL de la imagen principal.
min_price	integer	Precio mínimo.
total_stock	integer	Stock total.
has_stock	boolean	true si hay stock.

5.2.8 CartItem

Campo	Tipo	Descripción
id	string (uuid)	ID del item.
product_variant	object (ProductVariant)	Variante con datos del producto.
product	object (ProductListItem)	Producto resumido.
quantity	integer	Cantidad.
unit_price	integer	Precio unitario en centavos (snapshot).
subtotal	integer	quantity * unit_price.
created_at	integer (timestamp ms)	Fecha de creación.

5.2.9 Cart

Campo	Tipo	Descripción
id	string (uuid)	ID del carrito.
status	string	active, abandoned, converted.
items	array (CartItem)	Items del carrito.
subtotal	integer	Suma de subtotales.

Campo	Tipo	Descripción
total	integer	Total (= subtotal en MVP).
currency	string	Moneda (PEN).
item_count	integer	Cantidad total de items.
updated_at	integer (timestamp ms)	Última modificación.

5.2.10 OrderItem

Campo	Tipo	Descripción
id	string (uuid)	ID del item.
product_variant_id	string (uuid)	ID de la variante.
product_name	string	Nombre del producto (snapshot).
variant_details	object	{size, color, sku} (snapshot).
quantity	integer	Cantidad.
unit_price	integer	Precio unitario en centavos.
subtotal	integer	quantity * unit_price.

5.2.11 Payment

Campo	Tipo	Descripción
id	string (uuid)	ID del pago.
order_id	string (uuid)	ID del pedido.
amount	integer	Monto en centavos.
currency	string	Moneda.
status	string	pending, succeeded, failed, refunded.
method	string	card, yape, plin, cash.
stripe_payment_intent_id	string null	ID del PaymentIntent.
error_message	string null	Mensaje de error si falló.
created_at	integer (timestamp ms)	Fecha de creación.

5.2.12 Order

Campo	Tipo	Descripción
id	string (uuid)	ID del pedido.
customer_id	string (uuid) null	ID del cliente (null si es guest).
customer	object (User) null	Datos del cliente.
guest_email	string null	Email del invitado.
guest_name	string null	Nombre del invitado.
guest_phone	string null	Teléfono del invitado.
shipping_address	object (ShippingAddress)	Dirección de envío.
subtotal	integer	Subtotal en centavos.
total	integer	Total en centavos.
currency	string	Moneda.
status	string	pendiente, pagado, enviado, entregado, cancelado, fallido.
stripe_payment_intent_id	string null	ID del PaymentIntent.
items	array (OrderItem)	Items del pedido.
payment	object (Payment) null	Pago asociado.
notes	string null	Notas.
created_at	integer (timestamp ms)	Fecha de creación.
updated_at	integer (timestamp ms)	Última modificación.

5.2.13 CheckoutResponse

Campo	Tipo	Descripción
order	object (Order)	Orden creada.

Campo	Tipo	Descripción
client_secret	string	client_secret del PaymentIntent para Flutter Stripe.
payment_intent_id	string	ID del PaymentIntent.
publishable_key	string	Publishable key de Stripe (para inicializar el SDK).

5.2.14 StockMovement

Campo	Tipo	Descripción
id	string (uuid)	ID del movimiento.
product_variant_id	string (uuid)	ID de la variante.
type	string	sale, restock, adjustment, cancellation.
quantity	integer	Cantidad del cambio.
previous_stock	integer	Stock previo.
new_stock	integer	Stock nuevo.
reference_type	string null	Tipo de referencia.
reference_id	string (uuid) null	ID de la referencia.
notes	string null	Notas.
created_at	integer (timestamp ms)	Fecha.
created_by	string (uuid) null	Admin que registró.

5.2.15 StockAlert

Campo	Tipo	Descripción
id	string (uuid)	ID de la alerta.
product_variant_id	string (uuid)	ID de la variante.
variant	object (ProductVariant)	Variante con producto anidado.
threshold	integer	Umbral configurado.

Campo	Tipo	Descripción
current_stock	integer	Stock al momento de la alerta.
status	string	active, resolved.
created_at	integer (timestamp ms)	Fecha de generación.
resolved_at	integer (timestamp ms) null	Fecha de resolución.

5.2.16 DashboardMetrics

Campo	Tipo	Descripción
total_sales	integer	Ventas totales en centavos.
orders_count	integer	Número de pedidos (pagados+).
avg_ticket	integer	Ticket promedio (total_sales / orders_count).
currency	string	Moneda.
from	integer (timestamp ms)	Inicio del rango.
to	integer (timestamp ms)	Fin del rango.

5.2.17 TopProduct

Campo	Tipo	Descripción
product_id	string (uuid)	ID del producto.
product_name	string	Nombre.
main_image_url	string null	Imagen principal.
units_sold	integer	Unidades vendidas.
revenue	integer	Ingresos generados en centavos.

5.2.18 OrdersByStatus

Campo	Tipo	Descripción
status	string	Estado del pedido.
count	integer	Cantidad de pedidos.
total	integer	Suma de totales en centavos.

5.2.19 SalesTimelinePoint

Campo	Tipo	Descripción
date	string	Fecha del bucket (formato YYYY-MM-DD).
sales	integer	Ventas en centavos.
orders_count	integer	Número de pedidos.

5.3 Objetos comunes

5.3.1 Pagination

Campo	Tipo	Descripción
page	integer	Página actual (1-based).
limit	integer	Cantidad de registros por página.
total	integer	Total de registros.
total_pages	integer	Total de páginas.
has_next	boolean	true si hay página siguiente.
has_prev	boolean	true si hay página anterior.

5.3.2 SuccessResponse (envelope)

```
{  
  "success": true,
```



```
"data": {},  
"message": "Operación realizada correctamente."  
}
```

5.3.3 ErrorResponse (envelope)

```
{  
  "success": false,  
  "error": {  
    "code": "ERROR_CODE",  
    "message": "Descripción del error.",  
    "details": {}  
  }  
}
```

Campo	Tipo	Descripción
success	boolean	Siempre false en errores.
error.code	string	Código interno del error.
error.message	string	Mensaje legible.
error.details	object null	Detalles adicionales (ej. errores de validación por campo).

5.3.4 PaginatedResponse

```
{  
  "success": true,  
  "data": {  
    "items": [],  
    "pagination": {  
      "page": 1,  
      "limit": 20,  
      "total": 150,  
    }  
  }  
}
```

```

        "total_pages": 8,
        "has_next": true,
        "has_prev": false
    }
}
}

```

5.4 Enumeraciones

5.4.1 user_role

Valor	Descripción
admin	Administrador de la tienda.
customer	Cliente registrado.

5.4.2 order_status

Valor	Descripción
pendiente	Pedido creado, pago pendiente.
pagado	Pago confirmado por Stripe.
enviado	Pedido despachado.
entregado	Pedido entregado al cliente.
cancelado	Pedido cancelado.
fallido	Pago o stock fallido.

5.4.3 payment_status

Valor	Descripción
pending	Pago iniciado.

Valor	Descripción
succeeded	Pago exitoso.
failed	Pago fallido.
refunded	Pago reembolsado.

5.4.4 payment_method

Valor	Descripción
card	Tarjeta de crédito/débito.
yape	Yape.
plin	Plin.
cash	Contraentrega.

5.4.5 currency

Valor	Descripción
PEN	Soles peruanos.
USD	Dólares estadounidenses.

5.4.6 cart_status

Valor	Descripción
active	Carrito en uso.
abandoned	Carrito abandonado.
converted	Carrito convertido en pedido.

5.4.7 stock_movement_type

Valor	Descripción
sale	Venta (decremento automático).
restock	Reposición de stock.
adjustment	Ajuste manual.
cancellation	Cancelación de pedido.

5.4.8 stock_alert_status

Valor	Descripción
active	Alerta pendiente.
resolved	Alerta resuelta.

- Como todos los DTO utilizados por la API deben documentarse en esta sección.
- Como los endpoints deben reutilizar los DTO existentes.
- Como los cambios incompatibles requieren una nueva versión del contrato.

6 Formato estándar

6.1 Respuesta exitosa

```
{
  "success": true,
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  },
  "message": "Recurso creado correctamente."
}
```

6.2 Respuesta con error

```
{
  "success": false,
  "error": {
    "code": "INVALID_DATA",
    "message": "El email ya está registrado.",
    "details": {
      "field": "email",
      "value": "test@example.com"
    }
  }
}
```

6.3 Respuesta paginada

```
{
  "success": true,
  "data": {
    "items": [
      { "id": "...", "name": "..." }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 50,
      "total_pages": 3,
      "has_next": true,
      "has_prev": false
    }
  }
}
```

}

- Como todas las respuestas deben mantener la misma estructura de envelope.
- Como los errores deben ser claros y consistentes.

7 Endpoints

A continuación se documentan los **35 endpoints** de la API v1. Cada endpoint incluye: descripción, método, ruta, autenticación, roles, parámetros, headers, DTOs, códigos HTTP y reglas.

7.1 Health Check

7.1.1 [API-001] Health Check

Descripción

Endpoint de liveness check para verificar que el backend está operativo.

Método

GET

Ruta

/api/health

Autenticación: No

Roles permitidos: público

7.1.2 Códigos HTTP

Código	Descripción
200	Servicio operativo.

7.1.3 Response

```
{  
  "success": true,  
  "data": {  
    "status": "ok",  
    "service": "smartpyme-api",  
    "version": "1.0.0",  
    "timestamp": 1719496200000  
  }  
}
```

7.2 Autenticación

7.2.1 [API-002] Registro de usuario

Descripción

Registra un nuevo usuario (admin o customer). En single-tenant solo se permite un admin por despliegue.

Método: POST

Ruta: /api/auth/register

Autenticación: No

Roles permitidos: público

7.2.2 Request Body

DTO: RegisterRequest

7.2.3 Response 201

DTO: AuthResponse

7.2.4 Códigos HTTP

Código	Descripción
201	Usuario creado.
400	Datos inválidos (Zod).
409	Email duplicado.
422	Regla de negocio (ej. segundo admin).

7.2.5 Reglas

- Como la contraseña debe tener al menos 8 caracteres.
 - Como el email debe ser único.
 - Como solo puede existir un admin (RN-014).
 - Como el backend hashea la contraseña con bcrypt factor 10.
 - Como el JWT expira en 7 días.
-

7.2.6 [API-003] Inicio de sesión

Descripción

Autentica un usuario y devuelve un JWT.

Método: POST

Ruta: /api/auth/login

Autenticación: No

Roles permitidos: público

7.2.7 Request Body

DTO: LoginRequest

7.2.8 Response 200

DTO: AuthResponse

7.2.9 Códigos HTTP

Código	Descripción
200	Login exitoso.
400	Datos inválidos.
401	Credenciales inválidas.

7.2.10 Reglas

- Como el mensaje de error debe ser genérico (no revelar si el email existe).
- Como el token debe incluir role y userId en los claims.

7.2.11 [API-004] Usuario actual

Descripción

Devuelve los datos del usuario autenticado.

Método: GET

Ruta: /api/auth/me

Autenticación: Sí

Roles permitidos: customer, admin

7.2.12 Headers

Header	Obligatorio	Descripción
Authorization	Sí	Bearer <jwt>

7.2.13 Response 200

DTO: User

7.2.14 Códigos HTTP

Código	Descripción
200	Datos del usuario.
401	Token inválido o expirado.

7.3 Categorías (admin)

7.3.1 [API-005] Listar categorías

Descripción

Lista todas las categorías (activas e inactivas). Solo admin.

Método: GET

Ruta: /api/categories

Autenticación: Sí

Roles permitidos: admin

7.3.2 Query Params

Nombre	Tipo	Obligatorio	Descripción
page	integer	No	Página (default 1).
limit	integer	No	Items por página (default 20, max 100).
is_active	boolean	No	Filtrar por estado.
q	string	No	Búsqueda por nombre.

7.3.3 Response 200

DTO: PaginatedResponse<Category>

7.3.4 Códigos HTTP

Código	Descripción
200	Lista de categorías.
401	No autenticado.
403	No es admin.

7.3.5 [API-006] Crear categoría

Descripción

Crea una nueva categoría.

Método: POST

Ruta: /api/categories

Autenticación: Sí · **Roles:** admin

7.3.6 Request Body

DTO: CreateCategoryRequest

7.3.7 Response 201

DTO: Category

7.3.8 Códigos HTTP

Código	Descripción
201	Categoría creada.
400	Datos inválidos.
401	No autenticado.
403	No es admin.
409	Nombre duplicado.

7.3.9 Reglas

- Como el slug se genera automáticamente desde el nombre.

7.3.10 [API-007] Obtener categoría por ID

Descripción: GET /api/categories/:id · Auth admin

7.3.11 Path Params

Nombre	Tipo	Descripción
id	string (uuid)	ID de la categoría.

7.3.12 Response 200: Category · 404 si no existe.

7.3.13 [API-008] Actualizar categoría

Descripción

Actualiza parcialmente una categoría.

Método: PATCH

Ruta: /api/categories/:id

Auth: admin

7.3.14 Path Params: id (uuid)

7.3.15 Request Body: UpdateCategoryRequest

7.3.16 Response 200: Category

7.3.17 Códigos HTTP

Código	Descripción
200	Categoría actualizada.
400	Datos inválidos.
404	Categoría no encontrada.
409	Nombre duplicado.

7.3.18 [API-009] Eliminar categoría

Descripción

Elimina una categoría. Falla si tiene productos asociados.

Método: DELETE

Ruta: /api/categories/:id

Auth: admin

7.3.19 Path Params: id (uuid)

7.3.20 Response 204: Sin contenido.

7.3.21 Códigos HTTP

Código	Descripción
204	Eliminada.
404	No encontrada.
409	Categoría con productos asociados.

7.4 Productos (admin)

7.4.1 [API-010] Listar productos (admin)

Descripción

Lista todos los productos incluyendo inactivos. Solo admin.

Método: GET · **Ruta:** /api/products

Auth: admin

7.4.2 Query Params

Nombre	Tipo	Obligatorio	Descripción
page	integer	No	Default 1.
limit	integer	No	Default 20, max 100.
category_id	string (uuid)	No	Filtrar por categoría.
is_active	boolean	No	Filtrar por estado.
q	string	No	Búsqueda por nombre.
sort	string	No	name, price_asc, price_desc, newest.

7.4.3 Response 200: PaginatedResponse<ProductListItem>

7.4.4 [API-011] Crear producto

Descripción

Crea un nuevo producto sin variantes ni imágenes (se agregan después).

Método: POST · **Ruta:** /api/products · **Auth:** admin

7.4.5 Request Body: CreateProductRequest

7.4.6 Response 201: Product (con arrays vacíos de variants e images)

7.4.7 Códigos HTTP: 201, 400, 401, 403, 404 (categoría no existe)

7.4.8 Reglas

- Como `base_price > 0`.
 - Como el slug se genera automáticamente.
 - Como el producto se crea con `variants=[]` e `images=[]`.
-

7.4.9 [API-012] Obtener producto por ID

GET /api/products/:id · Auth admin

7.4.10 Response 200: Product (completo con variants e images)

7.4.11 Códigos HTTP: 200, 404

7.4.12 [API-013] Actualizar producto

PATCH /api/products/:id · Auth admin

7.4.13 Request Body: UpdateProductRequest

7.4.14 Response 200: Product

7.4.15 Códigos HTTP: 200, 400, 404

7.4.16 [API-014] Eliminar producto

DELETE /api/products/:id · Auth admin

7.4.17 Response 204: Sin contenido

7.4.18 Códigos HTTP: 204, 404

7.4.19 Reglas

- Como eliminar un producto hace CASCADE en variants, images y stock_movements.
 - Como eliminar un producto falla si tiene order_items (RESTRICT).
-

7.5 Variantes de producto

7.5.1 [API-015] Crear variante

POST /api/products/:id/variants · Auth admin

7.5.2 Path Params: id (uuid del producto)

7.5.3 Request Body: CreateVariantRequest

7.5.4 Response 201: ProductVariant

7.5.5 Códigos HTTP: 201, 400, 404, 409 (combinación duplicada)

7.5.6 Reglas

- Como (product_id, size, color) debe ser único.

- Como sku debe ser único globalmente.
 - Como stock ≥ 0 .
-

7.5.7 [API-016] Actualizar variante

PATCH /api/products/:id/variants/:variantId · Auth admin

7.5.8 Path Params: id (producto), variantId (variante)

7.5.9 Request Body: UpdateVariantRequest

7.5.10 Response 200: ProductVariant

7.5.11 Códigos HTTP: 200, 400, 404, 409

7.5.12 [API-017] Eliminar variante

DELETE /api/products/:id/variants/:variantId · Auth admin

7.5.13 Response 204

7.5.14 Códigos HTTP: 204, 404, 409 (variante con order_items)

7.6 Imágenes de producto

7.6.1 [API-018] Subir imagen

Descripción

Sube una imagen al storage R2 y la asocia al producto.

Método: POST · **Ruta:** /api/products/:id/images · **Auth:** admin

7.6.2 Request

multipart/form-data con campo file (JPG/PNG/WebP, máx 5 MB).

7.6.3 Query Params

Nombre	Tipo	Obligatorio	Descripción
position	integer	No	Posición de la imagen (default: siguiente disponible).

7.6.4 Response 201: ProductImage

7.6.5 Códigos HTTP

Código	Descripción
201	Imagen subida.
400	Formato o tamaño inválido.
404	Producto no existe.
502	Error al subir a R2.

7.6.6 Reglas

- Como solo se aceptan JPG, PNG, WebP.
- Como el tamaño máximo es 5 MB.
- Como se rechazan más de 5 imágenes por producto (RN derivado).

7.6.7 [API-019] Eliminar imagen

DELETE /api/products/:id/images/:imageId · Auth admin

7.6.8 Response 204

7.7 Catálogo público

7.7.1 [API-020] Listar productos del catálogo

Descripción

Endpoint público para que los clientes naveguen el catálogo. Solo devuelve productos activos con stock.

Método: GET · **Ruta:** /api/catalog/products

Auth: No

Roles permitidos: público

7.7.2 Query Params

Nombre	Tipo	Obligatorio	Descripción
page	integer	No	Default 1.
limit	integer	No	Default 20, max 100.
category_id	string (uuid)	No	Filtrar por categoría.
category_slug	string	No	Filtrar por slug de categoría.
q	string	No	Búsqueda por nombre (LIKE).
min_price	integer	No	Precio mínimo en centavos.
max_price	integer	No	Precio máximo en centavos.
in_stock	boolean	No	Solo con stock.
sort	string	No	name, price_asc, price_desc, newest.

7.7.3 Response 200: PaginatedResponse<ProductListItem>

7.7.4 Reglas

- Como solo se devuelven productos is_active=true.
- Como solo se devuelven productos con total_stock > 0 si in_stock=true.
- Como el ordenamiento por defecto es newest.

7.7.5 [API-021] Detalle de producto público

Descripción

Devuelve el detalle completo de un producto por slug. Público.

Método: GET · **Ruta:** /api/catalog/products/:slug

Auth: No

7.7.6 Path Params: slug (string)

7.7.7 Response 200: Product (completo)

7.7.8 Códigos HTTP: 200, 404

7.7.9 Reglas

- Como solo devuelve productos activos.
- Como 404 si el slug no existe o el producto está inactivo.

7.8 Carrito de compras

7.8.1 [API-022] Obtener carrito activo

Descripción

Devuelve el carrito activo del cliente o sesión. Si no existe, lo crea automáticamente.

Método: GET · **Ruta:** /api/cart

Auth: No (guest) o Sí (customer)

7.8.2 Headers (opcional para guest)

Header	Descripción
X-Session-Id	ID de sesión para guest. Si no se envía, el backend genera uno y lo devuelve en X-Session-Id del response.

7.8.3 Response 200: Cart

7.8.4 Códigos HTTP: 200, 500

7.8.5 Reglas

- Como si no hay carrito, se crea uno nuevo con `status='active'`.
- Como el `session_id` se persiste en el cliente (header).
- Como para clientes autenticados, se busca por `customer_id`.

7.8.6 [API-023] Agregar ítem al carrito

POST /api/cart/items · Auth opcional

7.8.7 Request Body: AddCartItemRequest

7.8.8 Response 200: Cart (con el nuevo ítem)

7.8.9 Códigos HTTP

Código	Descripción
200	Item agregado.
400	Datos inválidos.
404	Variante no existe.
422	Stock insuficiente.

7.8.10 Reglas

- Como si la variante ya está en el carrito, se suma la cantidad.

- Como se valida `quantity <= stock` de la variante.
 - Como se toma snapshot del precio unitario.
-

7.8.11 [API-024] Actualizar cantidad de un item

PATCH `/api/cart/items/:id` · Auth opcional

7.8.12 Path Params: id (uuid del cart_item)

7.8.13 Request Body: UpdateCartItemRequest

7.8.14 Response 200: Cart

7.8.15 Códigos HTTP: 200, 404, 422 (stock insuficiente)

7.8.16 [API-025] Eliminar item del carrito

DELETE `/api/cart/items/:id` · Auth opcional

7.8.17 Response 200: Cart

7.8.18 Códigos HTTP: 200, 404

7.8.19 [API-026] Vaciar carrito

DELETE `/api/cart` · Auth opcional

7.8.20 Response 200: Cart (vacío)

7.9 Checkout y pagos

7.9.1 [API-027] Iniciar checkout

Descripción

Crea una orden en estado pendiente, un PaymentIntent en Stripe y devuelve el `client_secret`.

Método: POST · **Ruta:** /api/checkout

Auth: No (guest) o Sí (customer)

7.9.2 Request Body: CheckoutRequest

7.9.3 Response 200: CheckoutResponse

7.9.4 Códigos HTTP

Código	Descripción
200	Checkout iniciado.
400	Datos inválidos.
422	Carrito vacío.
502	Error de Stripe.

7.9.5 Reglas

- Como rechaza si el carrito está vacío.
- Como valida coherencia del total (RN-012).
- Como crea orden con `status='pendiente'` y `stripe_payment_intent_id`.
- Como devuelve `client_secret` para que Flutter use `flutter_stripe`.
- Como el stock NO se descuenta aún (se hace al confirmar pago).

7.9.6 [API-028] Confirmar pago

Descripción

El cliente llama a este endpoint tras completar el pago en Flutter. El backend consulta a Stripe el estado del PaymentIntent y actualiza la orden + stock.

Método: POST · **Ruta:** /api/payments/confirm

Auth: No o Sí

7.9.7 Request Body: ConfirmPaymentRequest

7.9.8 Response 200: Order (actualizado con status='pagado' o fallido)

7.9.9 Códigos HTTP

Código	Descripción
200	Pago procesado.
404	Orden o PaymentIntent no encontrado.
422	Pago fallido según Stripe.
502	Error al consultar Stripe.

7.9.10 Reglas

- Como el backend consulta `stripe.paymentIntents.retrieve(id)` para verificar el estado real.
- Como si el pago fue exitoso, se actualiza `orders.status='pagado'` y `payments.status='succeeded'`.
- Como se decrementa el stock de cada variante con `stock_movements` atómicos.
- Como si el stock es insuficiente, se marca la orden como fallido y se reembolsa automáticamente vía Stripe.
- Como se vacía el carrito del cliente.
- Como si el pago falló, se devuelve 422 con detalles.

7.9.11 [API-029] Webhook de Stripe (reservado v2)

Descripción

Endpoint reservado para el webhook real de Stripe en una versión futura. En v1 las confirmaciones se hacen vía POST /api/payments/confirm.

Método: POST · **Ruta:** /api/payments/webhook · **Auth:** No (verificación de firma Stripe)

7.9.12 Request: Payload de Stripe

7.9.13 Response 200: { received: true }

7.9.14 Códigos HTTP: 200, 400 (firma inválida)

7.10 Pedidos

7.10.1 [API-030] Listar pedidos (admin)

GET /api/orders · Auth admin

7.10.2 Query Params

Nombre	Tipo	Obligatorio	Descripción
page	integer	No	Default 1.
limit	integer	No	Default 20.
status	string	No	Filtrar por estado.
from	integer (timestamp ms)	No	Fecha desde.
to	integer (timestamp ms)	No	Fecha hasta.
customer_id	string (uuid)	No	Filtrar por cliente.

7.10.3 Response 200: PaginatedResponse<Order>

7.10.4 Códigos HTTP: 200, 401, 403

7.10.5 [API-031] Obtener pedido por ID

GET /api/orders/:id · Auth admin o dueño (customer)

7.10.6 Response 200: Order

7.10.7 Códigos HTTP: 200, 401, 403 (no es dueño ni admin), 404

7.10.8 Reglas

- Como si el token es de customer, el pedido debe pertenecerle (customer_id debe coincidir).
- Como un customer no puede ver pedidos de otros clientes.

7.10.9 [API-032] Cambiar estado de pedido (admin)

PATCH /api/orders/:id/status · Auth admin

7.10.10 Request Body: UpdateOrderStatusRequest

7.10.11 Response 200: Order

7.10.12 Códigos HTTP

Código	Descripción
200	Estado actualizado.
400	Datos inválidos.
403	No es admin.
404	Pedido no existe.
422	Transición de estado inválida (RN-002).

7.10.13 Reglas

- Como la transición debe ser secuencial: pendiente → pagado → enviado → entregado.
- Como la cancelación solo es válida en pendiente o pagado.

- Como un cambio inválido devuelve 422 con código INVALID_STATUS_TRANSITION.
-

7.10.14 [API-033] Historial de pedidos del cliente

GET /api/customers/me/orders · Auth customer

7.10.15 Query Params: page, limit, status (opcional)

7.10.16 Response 200: PaginatedResponse<Order>

7.10.17 Códigos HTTP: 200, 401

7.10.18 Reglas

- Como solo devuelve los pedidos del cliente autenticado.
 - Como no permite filtrar por customer_id (se toma del token).
-

7.11 Inventario (admin)

7.11.1 [API-034] Listar alertas de stock

GET /api/inventory/alerts · Auth admin

7.11.2 Query Params: page, limit, status (default active)

7.11.3 Response 200: PaginatedResponse<StockAlert>

7.11.4 Códigos HTTP: 200, 401, 403

7.11.5 [API-035] Listar movimientos de stock

GET /api/inventory/movements · Auth admin

7.11.6 Query Params: page, limit, type, product_variant_id, from, to

7.11.7 Response 200: PaginatedResponse<StockMovement>

7.11.8 Códigos HTTP: 200, 401, 403

7.11.9 [API-036] Registrar movimiento manual de stock

POST /api/inventory/movements · Auth admin

7.11.10 Request Body: CreateStockMovementRequest

7.11.11 Response 201: StockMovement

7.11.12 Códigos HTTP: 201, 400, 404, 422 (stock negativo)

7.11.13 Reglas

- Como crea un stock_movement con type='restock' o 'adjustment'.
 - Como actualiza el stock de la variante en transacción atómica.
 - Como si new_stock < 0, se rechaza con 422.
-

7.12 Dashboard (admin)

7.12.1 [API-037] Métricas generales

GET /api/dashboard/metrics · Auth admin

7.12.2 Query Params

Nombre	Tipo	Obligatorio	Descripción
from	integer (timestamp ms)	No	Inicio del rango (default: hace 30 días).
to	integer (timestamp ms)	No	Fin del rango (default: ahora).

7.12.3 Response 200: DashboardMetrics

7.12.4 Códigos HTTP: 200, 401, 403

7.12.5 Reglas

- Como solo cuenta pedidos con status IN ('pagado', 'enviado', 'entregado').

7.12.6 [API-038] Top productos vendidos

GET /api/dashboard/top-products · Auth admin

7.12.7 Query Params: from, to, limit (default 5, max 20)

7.12.8 Response 200: TopProduct[]

7.12.9 [API-039] Pedidos por estado

GET /api/dashboard/orders-by-status · Auth admin

7.12.10 Query Params: from, to

7.12.11 Response 200: OrdersByStatus[]

7.12.12 [API-040] Serie temporal de ventas

GET /api/dashboard/sales-timeline · Auth admin

7.12.13 Query Params

Nombre	Tipo	Obligatorio	Descripción
from	integer (timestamp ms)	Sí	Inicio del rango.
to	integer (timestamp ms)	Sí	Fin del rango.
granularity	string	No	day (default), week, month.

7.12.14 Response 200: SalesTimelinePoint[]

7.13 Resumen de endpoints

Módulo	Endpoints	Auth
Health	1	No
Auth	3	Mixto
Categories	5	Admin
Products	5	Admin
Variants	3	Admin
Images	2	Admin
Catalog	2	No
Cart	5	Opcional
Checkout/Payments	3	Mixto

Módulo	Endpoints	Auth
Orders	4	Mixto
Inventory	3	Admin
Dashboard	4	Admin
Total	40	—

8 Códigos de error

Código	HTTP	Descripción
INVALID_DATA	400	Datos inválidos (Zod validation failed).
INVALID_JSON	400	Body no es JSON válido.
MISSING_FIELD	400	Campo obligatorio faltante.
UNAUTHORIZED	401	Sin token o token inválido.
TOKEN_EXPIRED	401	JWT expirado.
FORBIDDEN	403	Sin permisos para el recurso.
NOT_FOUND	404	Recurso no encontrado.
RESOURCE_INACTIVE	404	Recurso existe pero está inactivo.
CONFLICT	409	Conflicto (duplicado, estado inconsistente).
EMAIL_ALREADY_EXISTS	409	Email ya registrado.
CATEGORY_HAS_PRODUCTS	409	Categoría no eliminable.
VARIANT_DUPLICATE	409	Combinación talla+color duplicada.
BUSINESS_RULE_VIOLATION	422	Regla de negocio violada.
EMPTY_CART	422	Carrito vacío.
INSUFFICIENT_STOCK	422	Stock insuficiente.
INVALID_STATUS_TRANSITION	422	Transición de estado no permitida.
PAYMENT_FAILED	422	Pago fallido.
RATE_LIMIT_EXCEEDED	429	Demasiadas peticiones.
INTERNAL_ERROR	500	Error interno del servidor.

Código	HTTP	Descripción
STRIPE_ERROR	502	Error al comunicarse con Stripe.
STORAGE_ERROR	502	Error al comunicarse con R2.

- Como todos los errores deben utilizar un código interno único.
- Como el frontend debe poder identificar el error únicamente mediante el código.

9 Versionado

- La API mantiene el prefijo `/api/` (sin `/v1/`) durante todo el ciclo de vida `v1`.
- Cambios compatibles hacia atrás (adicionar campo opcional, nuevo endpoint, nuevo query param) se aplican sin incremento de versión.
- Cambios incompatibles (eliminar campo, renombrar endpoint, cambiar tipo) requieren una nueva rama y migración del cliente.
- Política de deprecación: cuando un endpoint se vaya a eliminar, se mantiene funcional durante al menos 30 días con un header `Deprecation: true` y un mensaje en `error.message`.
- Las versiones se etiquetan con tags de Git en el backend (`v1.0.0`, `v1.1.0`, etc.).
- Como los cambios incompatibles deben generar una nueva versión del backend.
- Como las versiones anteriores deben mantenerse durante el periodo definido por el proyecto.

10 Historial de cambios

Versión	Fecha	Cambio
v1.0.0	Junio 2026	Contrato inicial. 40 endpoints. Documentación OpenAPI 3.1 + Swagger UI.

- Como toda modificación del contrato debe registrarse en este historial.

- Como frontend y backend deben utilizar la misma versión del contrato.

11 Riesgos técnicos

ID	Riesgo	Impacto	Probabilidad	Mitigación
API-001	Inconsistencia entre contrato y modelo de datos ante cambios	Alto	Media	Revisión cruzada manual entre API.md y MODELOS-DATOS.md en cada PR. Generar DTOs de Zod desde el schema Drizzle cuando sea posible.
API-002	Divergencia entre JSON del backend y DTOs de Flutter	Alto	Alta	Usar @hono/zod-openapi para generar OpenAPI 3.1 desde schemas Zod. Generar modelos Freezed en Flutter desde OpenAPI con openapi_generator.
API-003	CORS permisivo en producción	Alto	Baja	Variable de entorno CORS_ORIGIN permite restringir en producción. Documentar en .env.example.

ID	Riesgo	Impacto	Probabilidad	Mitigación
API-004	Sin rate limit puede permitir abuso en endpoints públicos	Medio	Media	Aceptado en MVP. Mitigar con Cloudflare o proxy inverso en producción. Implementar rate limit en endpoints /api/auth/* como mínimo.
API-005	Tamaño de respuestas sin compresión	Bajo	Alta	Habilitar gzip en el middleware de Hono. Configurar Accept-Encoding: gzip en cliente.
API-006	Falta de versionado en URL dificulta coexistencia de versiones	Bajo	Baja	Aceptado en MVP. Si se requiere v2, se introduce /api/v2/ con el router correspondiente.

- Como los cambios incompatibles deben detectarse antes del desarrollo.
- Como los riesgos de seguridad deben mitigarse antes del despliegue.

12 Criterios de aceptación

- Como todos los endpoints definidos en ALCANCE.md deben encontrarse documentados en este contrato.
- Como todos los DTOs utilizados por la API deben estar definidos en la sección de objetos compartidos.
- Como todos los códigos HTTP y errores posibles deben encontrarse documentados.

- Como el frontend debe poder desarrollar la integración únicamente utilizando este documento.
- Como el backend debe implementar exactamente el contrato definido (cero desviaciones).
- Como cualquier modificación del contrato debe actualizar este documento antes de implementarse.
- Como el contrato debe mantenerse sincronizado con `MODEL05-DAT05.md` cuando existan cambios estructurales.
- Como la especificación OpenAPI debe ser válida y la UI de Swagger debe cargar sin errores.
- Como cada endpoint debe tener al menos un criterio de éxito y uno de error documentado.
- Como los códigos de error deben ser únicos y estables para que el cliente pueda manejarlos sin ambigüedad.